

Proof of Concept Specification

AI-Powered Outage Management & Analysis System

Document ID: SPEC-AI-OM-2023-001 | **Version:** 1.0 Draft

Date: October 26, 2023 | **Author:** Technical Architecture Team

1. Executive Summary

This document outlines the technical specification for a Proof of Concept (PoC) designed to demonstrate the capabilities of an AI-driven system integrated with the Model Context Protocol (MCP). The proposed solution aims to revolutionize the existing "Call to Work" (C2W) process by automating incident analysis, reducing Mean Time to Resolution (MTTR), and preventing recurrent outages.

The PoC will simulate the ingestion and analysis of three distinct incident types—a novel issue, a repeat issue, and a complex cascading failure—to validate the AI's ability to provide actionable insights, draft Root Cause Analysis (RCA) reports, and suggest remediation steps in real-time.

2. Demo Overview & Objectives

The demo will feature a dashboard interface connected to a simulated backend powered by Large Language Models (LLMs) via MCP. It will showcase the lifecycle of a C2W ticket from creation to resolution support.

Primary Objectives:

- Validate Pattern Recognition:** Demonstrate the AI's ability to distinguish between new anomalies and historical patterns.
- Automate Context Assembly:** Show how MCP tools can aggregate logs, metrics, and code changes automatically.
- Reduce Cognitive Load:** Prove that AI-generated summaries and RCAs reduce the investigative burden on engineering teams.

3. Sample C2W Incident Scenarios

Scenario 1: New Issue (Payment Gateway Timeout)

Context: A sudden spike in 504 errors on the checkout service. No prior history of this specific failure pattern.

Attribute	Details
C2W ID	C2W-2023-8891
Severity	CRITICAL (SEV-1)
Symptoms	Checkout service API returning HTTP 504 Gateway Timeout. Users unable to complete transactions.
Simulated Logs	[ERROR] 2023-10-26 14:15:01.233 [CheckoutService] PaymentProviderClient: Request timeout after 30000ms. Upstream: https://api.payment-provider-x.com/v1/charge Context: OrderID=99123, UserID=5541

Scenario 2: Repeat Issue (DB Connection Pool Exhaustion)

Context: The Inventory Service becomes unresponsive. This has happened twice before due to unclosed JDBC connections in legacy code.

Attribute	Details
C2W ID	C2W-2023-9004
Severity	MAJOR (SEV-2)
Simulated Logs	[FATAL] 2023-10-26 15:30:10.551 [InventoryService] HikariPool-1 - Connection is not available, request timed out after 3000ms. Active: 50, Idle: 0, Waiting: 145. SQLState: 08001
Historical Match	Matches fingerprints of C2W-2023-7500 and C2W-2023-6200 (98% similarity score).

Scenario 3: Cascading Failure (CDN -> API -> DB)

Context: Complex chain reaction. A CDN failure causes clients to retry aggressively, overloading the API Gateway, which saturates the Database CPU.

Attribute	Details
C2W ID	C2W-2023-9150
Severity	CRITICAL (SEV-1)
Sequence	1. 16:00: CDN Health Check Fails (Region US-EAST) 2. 16:02: Traffic spike (400% increase) on Origin API due to retry storm

4. AI Analysis Workflow

Workflow for Scenario 1 (New Issue)

1. **Ingestion:** AI agent receives C2W alert via Webhook.
2. **Data Gathering (MCP):**
 - Agent queries `logs-tool` for "CheckoutService" errors in last 15 mins.
 - Agent queries `deployment-tool` to check for recent code changes (Result: None in last 24h).
 - Agent queries `vendor-status-tool` (Result: Payment Provider X reports partial outage).
3. **Analysis:** AI correlates internal 504s with external vendor status.
4. **Output:** Recommendation to enable "Circuit Breaker" pattern and switch to backup payment provider.

Workflow for Scenario 2 (Repeat Issue)

1. **Ingestion:** AI processes error signature "HikariPool-1 Connection not available".
2. **Pattern Matching:** Vector search against Historical Incident Database (RAG).
3. **Identification:** Identifies C2W-7500 as "Exact Match".
4. **Output:**
 - **Immediate Fix:** "Restart Inventory Service Pods (Temporary Relief)."
 - **Root Cause:** "Known leak in class `InventoryDAO.java`. Permanent fix pending in PR #402."

5. Technical Architecture

MCP Server Components

The architecture relies on the Model Context Protocol to bridge the LLM with infrastructure tools.

Component	Role	Underlying Tech
Orchestrator Agent	Central brain managing the incident lifecycle	GPT-4 / Claude 3.5 Sonnet
MCP: Observability	Fetches metrics, logs, traces	Datadog / Prometheus API
MCP: Knowledge Base	Retrieves past incidents, runbooks	Pinecone (Vector DB) / Confluence API

Component	Role	Underlying Tech
MCP: Source Control	Analyzes diffs and blame	GitHub / GitLab API

6. UI/UX Mockup Descriptions

VIEW 1: INCIDENT ANALYSIS DASHBOARD

Layout: Three-column layout.

- **Left Panel (Context):** Displays C2W ID, Severity, Timeline, and Active Team Members (pulled from Teams).
- **Center Panel (AI Investigation):** A chat-like interface showing the AI's thought process.

"Analyzing logs... Found 5000+ errors."

"Checking deployments... No recent changes."

"Comparing with history... Match found (85%)."
- **Right Panel (Actionable Intel):**

Root Cause Probability: Database Lock (90%), Network (10%)

Suggested Runbook: RB-DB-UNLOCK-PROCESS

Button: "Generate Draft RCA"

VIEW 2: HISTORICAL MATCHING CARD (SCENARIO 2)

Visual Element: A comparison card displayed when a repeat issue is detected.

Current Incident (C2W-9004) Error: Connection Timeout Service: Inventory Time: 15:30 EST	== 98% ==	Past Incident (C2W-7500) Error: Connection Timeout Service: Inventory Resolution: Restart Pods
--	--	--

7. Data Flow Description

The system operates on a rigorous event-driven architecture:

Flow A: Incident Ingestion

- Monitoring System (e.g., Splunk) triggers alert -> **Outage Management Tool** creates C2W Ticket.
- C2W Creation Event webhook fires to -> **AI Orchestrator Service**.

Flow B: Context Assembly (The "Loop")

1. **AI Orchestrator** parses the ticket description.
2. **AI Orchestrator** calls **MCP Server: Logs** with parameters `{service: 'checkout', window: '15m'}`.
3. **MCP Server: Logs** returns structured log snippets.
4. **AI Orchestrator** generates embedding vector for the incident signature.
5. **AI Orchestrator** calls **MCP Server: Vector DB** to find nearest neighbors (historical incidents).

8. Success Metrics & KPIs

The following metrics will define the success of the PoC:

Metric	Target	Measurement Method
Pattern Recognition Accuracy	> 90%	Percentage of "Repeat" incidents correctly identified vs. manual tagging.
Time to Hypothesis (TTH)	< 2 minutes	Time from C2W creation to first AI-generated root cause suggestion.
RCA Draft Quality	4/5 Rating	Subjective rating by Incident Commanders on utility of AI-drafted text.

9. Implementation Timeline

- **Week 1: Setup & Integration** - Configure MCP servers, connect dummy data sources for the 3 scenarios.
- **Week 2: Model Tuning** - Prompt engineering for the specific log formats and "Corporate Tone" for RCAs.
- **Week 3: Interface Development** - Build the Dashboard UI to visualize the AI workflow.
- **Week 4: Testing & Demo Prep** - Dry runs of the 3 scenarios, recording demo videos.

10. Technical Requirements

Hardware/Infrastructure:

- Containerized Environment (Docker/Kubernetes) for hosting MCP servers.
- Access to LLM API (OpenAI GPT-4 or Anthropic Claude via API Key).
- Vector Database instance (Pinecone or local ChromaDB) for historical matching.

Data Privacy:

- PII Sanitization layer must be active before logs are sent to the LLM.
- All simulated data for the PoC will be synthetic (mock user IDs, mock IPs).